target_setter Documentation

Release 1.0

Chheng Lydiya

Mar 10, 2026

CONTENTS

Welcome to the official documentation for **Target Setter**. This documentation provides system overview, installation instructions, technical references, and theoretical background used in the project.

CHAPTER

ONE

REPOSITORY

- GitHub: https://github.com/c-lydia/r2_ws.git

CHAPTER

TWO

CORE DOCUMENTATION

2.1 Introduction

2.1.1 Background

Motivation

Following the previous year in Android application development for robotics system, we have been using Android app as the controller for the robots.

However, this approach works with the xbox gamepad that acts as the main controller for our robots, using the buttons and joystick to navigate the robots manually. While this method provides real-time control, it limits automation and precise waypoint navigation. The initial idea of this project originated from Mr.PHOLL Chanphayuk, who proposed an Android-based interface that allows users to interact with the robot through touch inputs on the screen, combined with odometry feedback and a virtual joystick. The concept was intended for interactive tasks, such as robot basketball playing.

After changes in term members and project direction, the focus of the project evolved from manual joystick control to a waypoint-based navigation system. This shift was encouraged by Dr.SRANG Sarot, aiming to reduce dependency on gamepad controllers and to explore a more autonomous navigation approach using Android applications as the main interface.

Problem Statement

Manual control using gamepads or joysticks presents several limitations in robotic navigation systems:

- Continuous use input is required, increasing operator workload
- Precise positioning is difficult due to human reaction time and control sensitivity
- The system lacks scalability for autonomous behaviors

To address these issues, a waypoint navigation system is proposed, where uses can define target positions (waypoints) through an Android application, and the robot navigates to these positions using odometry and orientation data.

However, implementing such a system introduces challenges, including:

- Accurate coordinate transformation between Android interface and the robot's reference frame
- Proper handling of robot orientation (yaw) in navigation calculations
- Reliable communication between the Android application and the robot's ROS2-based control system
- Managing waypoint editing, execution, and return behaviors without causing state conflicts

These challenges motivated the need to redesign the calculation logic and communication protocol to ensure compatibility with ROS2 and improve navigation accuracy.

Literature Review

Waypoint-based navigation is a common approach in mobile robotics, especially in autonomous systems using frameworks such as ROS and ROS2. Previous works and standard robotics literature describe waypoint navigation as a method where a robot moves through a sequence of pre-defined target positions using localization and motion control algorithms.

Odometry-based navigation is widely used for estimating a robot's position by integrating wheel encoder data over time. Although odometry is simple to implement, it is sensitive to drift and orientation errors, particularly in yaw estimation. Many systems combine odometry with higher-level navigation logic to mitigate these issues.

Mobile interfaces for robot control, including Android-based applications, have also been explored as flexible and user-friendly solutions. Touch-based interfaces allow intuitive interaction, but require careful mapping between screen coordinates and real-world robot motion.

This project builds upon these established concepts, focusing on practical implementation rather than theoretical optimization.

Concepts

The waypoint navigation system in this project is based on the following key concepts:

- Waypoint navigation: the robot is instructed to move toward pre-defined target positions instead of being continuously controlled by the user.
- Odometry: the robot's position and orientation are estimated using incremental motion data.
- Yaw orientation: the robot's heading angle is critical for accurate navigation and must be correctly handled in coordinate transformations.
- Client-sever communication: the Android application acts as a client that sends waypoints data to the robot, while the robot processes these commands within a ROS2 environment.

For simplification, the system assumes:

- A planar (2D) environment
- Reliable odometry data without sensor fusion
- Pre-defined motion control handled by the robot's existing control stack

The Android application focuses on target setting, waypoint management, and command transmission, while the robot handles low-level motion execution.

2.1.2 Objective

This project is developed in order to:

- Design and implement Android app to define, edit, and manage navigation waypoints on a pre-defined map
- Develop custom ROS2 control logic to receive waypoint data and execute navigation commands on the robot
- Implement accurate coordinate transformation and scaling between Android interface and the robot's reference frame
- Handle robot orientation (yaw) correctly during waypoint navigation to improve positioning accuracy
- Establish reliable client-server communication between the Android app and the ROS2-based robot system
- Implement waypoint execution features including sequential navigation, editing, and return-to-previous-waypoint behaviors
- Test and evaluate system performance, identifying limitations related to map alignment, yaw accuracy, and state management.

2.1.3 Scope of work

This project covers:

- Android application development (Target_Setter)
- Communication system
- Robotic software development (ROS2)
- System integration and testing

This project does not include:

- Full autonomous robot navigation
- Sensor fusion beyond basic odometry and IMU integration
- Algorithm optimization
- Low-level communication protocol
- Hardware-level motor driver design
- Network security mechanism

2.2 Hardware Specification

This section outlines the hardware components and specifications for the mobile robot platform used in this project. The hardware selection is based on the requirements for waypoint navigation, odometry estimation, and client-server communication with an Android application.

2.2.1 Robot platform

- **Type:** omnidirectional mobile robot with mecanum wheels (holonomic motion)
- **Dimensions:** 0.5 m x 0.5 m x 0.3 m (approximately)

2.2.2 Sensors

The robot is equipped with 4 independent DC motors, driving 4 mecanum wheels with encoders attached for feedback. To compenstate for yaw estimation, an IMU (Inertial Measurement Unit) is included, providing angular velocity and linear acceleration data.

2.2.3 Drive system

- **Motors:** 4 DC motors, each driving on mecanum wheel
- **Motor specifications:** 24 V, 200 W max, integrated encoders
- **Motor drivers:** custom-built smart driver boards, each supporting 2 motors
- **Wheel type:** mecanum 0.06 radius (approximately)
- **Feedback:** quadrature encoders for wheel odometry

2.2.4 Sensors

Inertial Measurement Unit (IMU): provides angular velocity and linear acceleration

- **Model:** HFI-A9
- **Sampling rate:** 100 Hz

Encoder: integrated with each motor; used for odometry estimation

2.2.5 Compute hardware

High-level processor: NVIDIA Jetson Xavier

- **OS:** Ubuntu 20.04
- **Middleware:** ROS2 Foxy
- **Cooling:** cooling fan

Low-level controller: STM32 microcontroller

- Interfaces with motor driver and encoders for real-time control

Communication: CAN bus at 115200 bps

2.2.6 Network infrastructure

- **High-level control:** UDP of Wi-Fi between Android application and Jetson
- **IP configuration:** dynamic IP over DHCP
- **Indoor deployment:** Wi-Fi network required
- **Security:** basic
- **Remote access:** SSH for development and debugging purposes

2.2.7 Power system

Motors: 24 V Li-ion battery pack, DC-DC converter (12 V) for motor drivers

Electronics: 12 V Li-ion battery pack, voltage regulator for Jetson, sensors, and STM32

Safety features:

- Fuses on smart driver boards
- Overcurrent protection
- Emergency stop push button

2.2.8 Safety constraints

Hardware-level safety: fuses and emergency stop

Software-level safety:

- Speed limits enforced in control software
- Communication failure handling
- Watchdog timers for microcontroller to prevent system hang

2.2.9 Notes

1. Ensure CAN bus termination is correct to prevent communication errors
2. Confirm battery voltages before operation; over/under-voltage may damage electronics
3. When adding additional sensors, update ROS2 launch files for proper integration
4. For software testing, always use the emergency stop and verify motor driver fuses

2.3 Installation and Configuration

This section provides a comprehensive guide to installing and configuring the ROS2-based mobile manipulation app. It covers the necessary prerequisites, workspace setup, build instructions, configuration parameters, and tips for running the application effectively.

2.3.1 Prerequisites

- ROS2 installed on Ubuntu (any version is fine)
- Jetson Nano setup with ROS2 and necessary drivers for the robot hardware
- Android Studio installed for app development
- Basic familiarity with ROS2 concepts and Android development
- Network configuration for communication between the Android app and the robot
- Python 3 and necessary ROS2 Python packages for the robot nodes

2.3.2 Workspace Setup

1. Create a ROS2 workspace on the Jetson Nano:

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build
source install/setup.bash
```

2. Clone the project repository into the `src` directory:

```
cd ~/ros2_ws/src
git clone <repository_url>
```

3. Build the workspace:

```
cd ~/ros2_ws
colcon build
source install/setup.bash
```

4. Set up the Android project in Android Studio by importing the project from the cloned repository.

2.3.3 Configuration

- Update the ROS2 node parameters in the configuration files located in the *config* directory of the ROS2 package. This includes setting the correct IP address for communication and any robot-specific parameters.
- Ensure that the Android app's network settings are configured to communicate with the robot's IP address
- Adjust the waypoint navigation parameters such as speed limits, waypoint tolerance, and control gains as needed for your specific robot and environment.

2.3.4 Running the Application

1. Start the ROS2 nodes on the Jetson Nano:

```
source ~/ros2_ws/install/setup.bash
ros2 run <package_name> <package_node> # repeat for all necessary nodes
```

2. Run the Android application from Android Studio on a connected device or emulator. Or you can just open the `.apk` file on the phone directly.
3. Use the app interface to set waypoints and observe the robot's behavior as it navigates to the specified locations.

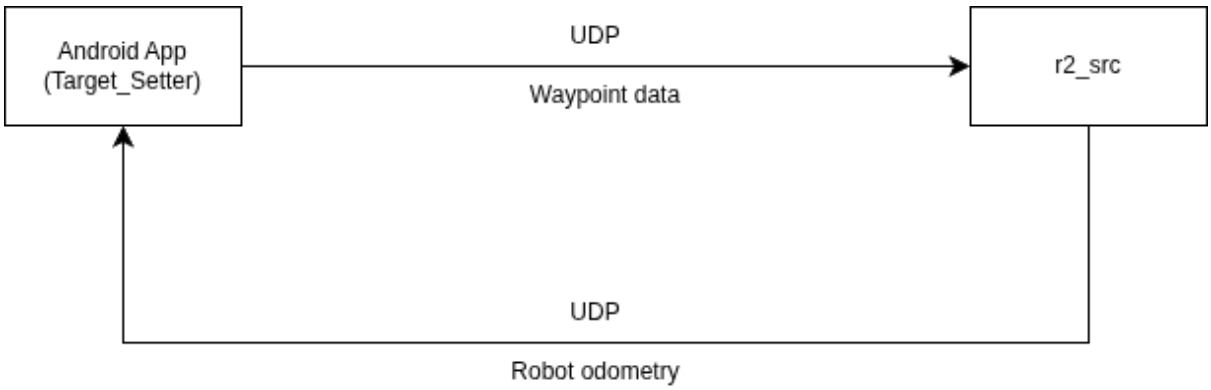
> **Attention**
>
> This release doesn't have launch file yet. Launch file will be added in the next update release.

2.3.5 Common Setup Pitfalls

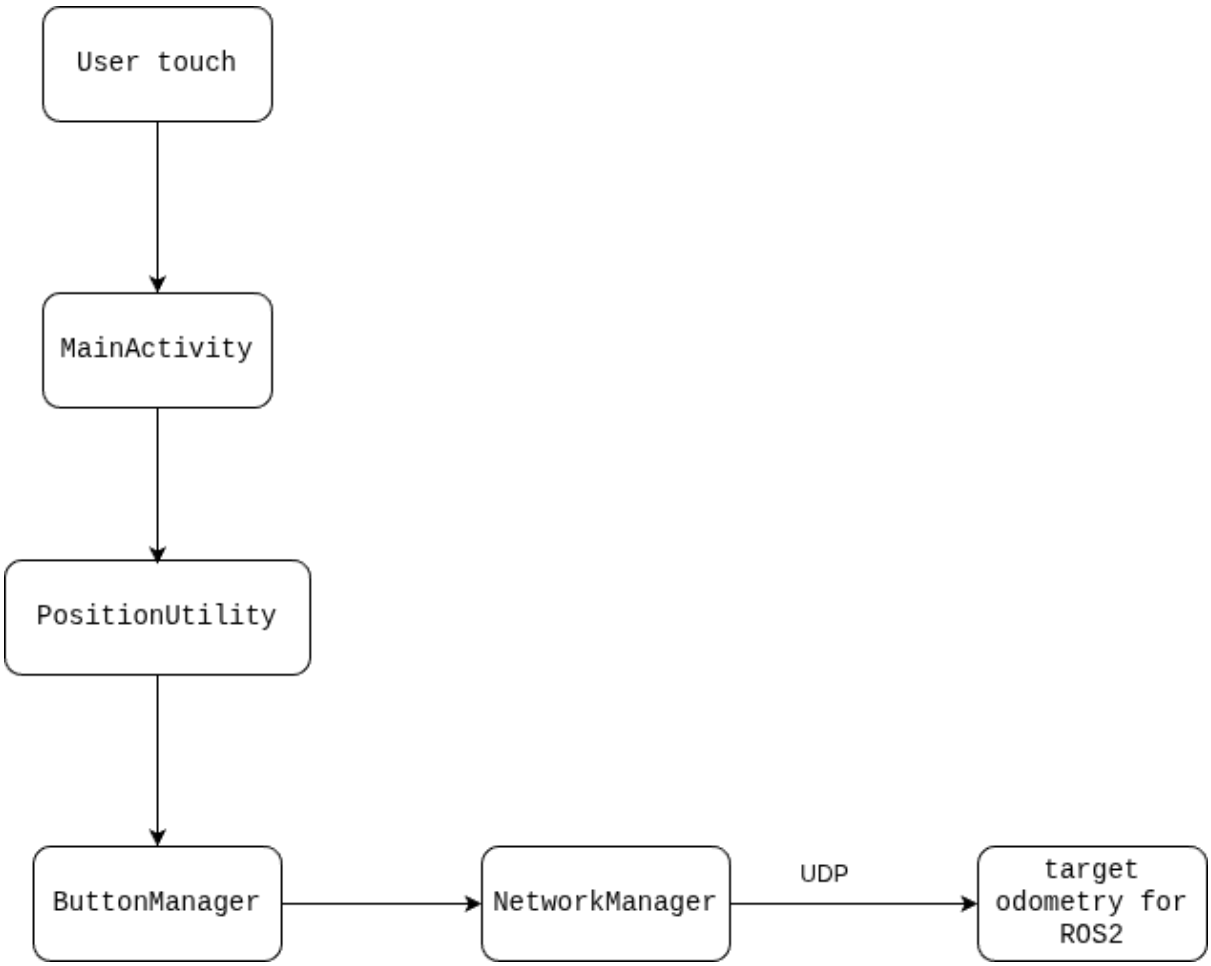
- Incorrect IP address configuration leading to communication failures between the app and the robot.
- Missing dependencies or incorrect ROS2 environment setup on the Jetson Nano.
- Incompatible ROS2 versions between the robot nodes and the app's communication protocol.
- Issues with the robot's hardware drivers or sensor configurations that affect navigation performance.
- Network connectivity issues, such as firewall settings or Wi-Fi interference, that disrupt communication between the app and the robot.

2.4 Technical Reference

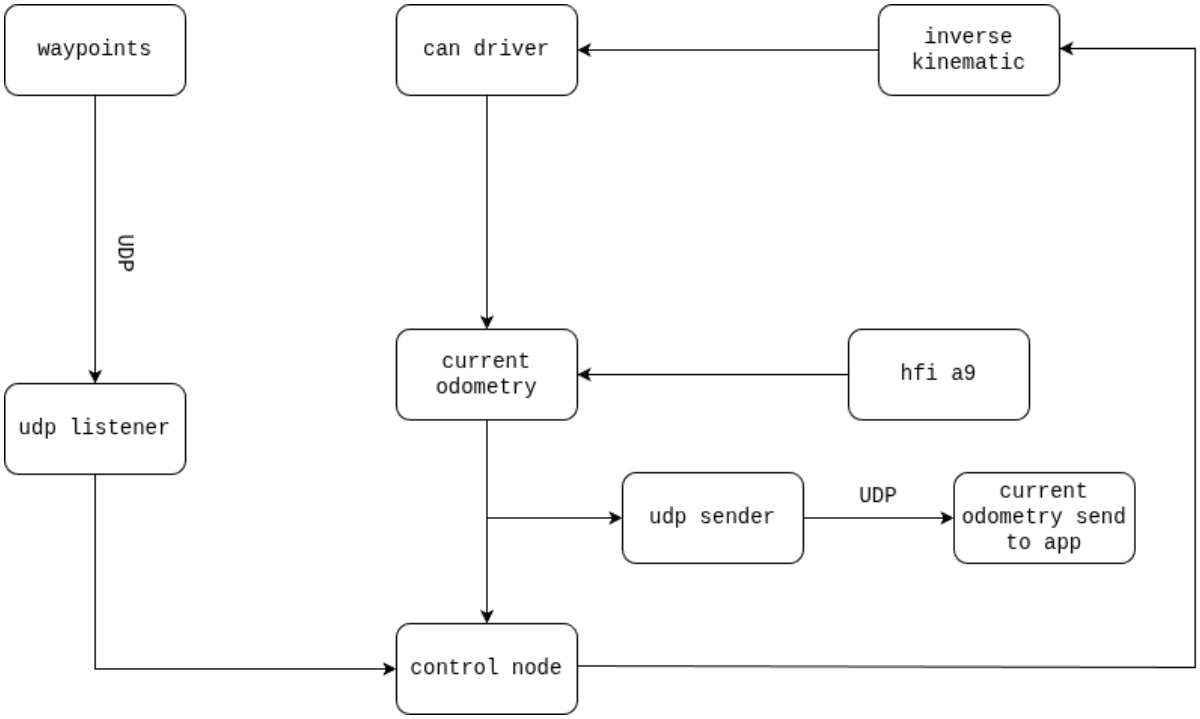
2.4.1 Architecture



Target_Setter app



r2_src



2.4.2 Waypoint queue and state machine

Waypoint queue

- **Waypoint queue:** stores received waypoints from */waypoint*
- **Active target:** currently executing waypoint
- **Visited waypoints:** stack of completed waypoints for return

2.4.3 State machine

- `IDLE`: no active target, waiting for waypoint or return command
- `NAVIGATING`: moving to `active_target`
- `EDIT`: active target is being edited
- `PAUSED`: robot is paused by keyboard or automatic pause
- `RETURNING`: moving to last visited waypoint (return command)

2.4.4 Communication protocol

UDP packet structure

Waypoints packet

- **Purpose:** send a list of waypoints for the robot to follow
- **Frequency:** send when pressing Send in the app

Packet Structure (binary):

Offset	Size (bytes)	Description
0	4	Header (0xAA)
4	4	Number of waypoints (counter)
8	4	Plan ID (version)
12	16 * N	Waypoint (X, Y) in double (8 B)

- N = number of waypoints
- Coordinates are in meters, converted from percentage position
- Byte order: Little Endian

Edit waypoint packet

- **Purpose:** update a previously sent waypoint
- **Frequency:** send when user edits a waypoint via the app

Packet structure (binary):

Offset	Size (bytes)	Description
0	1	Edited flag (1 = edited)
1	4	Waypoint index (1-based)
5	4	Plan ID
9	8	New X coordinate (double)
17	8	New Y coordinate (double)

Return message packet

- **Purpose:** send a single boolean signal to the robot
- **Frequency:** send when return button is pressed

Packet structure (binary):

Offset	Size (bytes)	Description
0	1	Edited flag (0=false,1=true)

Odometry data reception

- **Purpose:** receives current robot odometry
- **Frequency:** 10Hz

Packet structure (binary):

Offset	Size (bytes)	Description
0	8	X position (double)
8	8	Y position (double)
16	8	Yaw (double, radians)

- Byte order: Little Endian
- Latest odometry stored in memory for waypoint calculations

UDP data flow

Sending data

- App uses `NetworkManager.sendWaypoints()`, `sendEdit()`, and `sendMesage()`
- Waypoints are converted to binary packets before sending
- Each send operation is done in a separate thread
- Plan ID differentiates waypoint plans
- Packets sent via UDP to robot IP/port (default 5050)

Receiving data

- Persistent UDP socket listens (`startReceiving(port)`)
- Robot sends odometry at 10Hz (X, Y, Yaw)
- Incoming packets converted using Little Endian byte order
- Latest odometry stored in `latestOdometry`

Processing data

When adding waypoint:

- Convert touch position to normalized coordinates
- Compute target X, Y in meters
- Add to waypoint list, update UI
- Send waypoint via UDP

When editing waypoint:

- Specify index and new coordinates
- Update list and redraw UI
- Send edit packet via UDP with plan ID

Return message:

- Single-byte packet sent to trigger return behavior

Odometry:

- Display robot position
- Compute accurate target positions

2.4.5 Coordinate convention and control logic

Coordinate flow

Touch → UI → App → Robot

```
Touch Event (screen pixels)
   ↓ absolutePosition()
Absolute Coordinates relative to boundaryView
   ↓ normalizedPosition()
Normalized Coordinates (0-1)
   ↓ percentagePosition()
Percentage Coordinates (0-100%)
   ↓ targetPosition()
Global Frame (meters) → UDP → Robot
```

Robot → App (Odometry)

```
Odometry Packet (X, Y, yaw in meters) → inputPosition()
   ↓ metersToPxX/Y
Pixels on UI → Display robot position
```

Frame definition

Local frame:

- Origin: the center of the robot
- +x: forward
- +y: left
- +yaw: counter-clockwise

Global frame:

- Origin: top-left of the screen on the target_setter app
- +x: rightward on the screen on the target_setter app
- +y: downward on the screen on the target_setter app
- Waypoints that is sent to the robot is in global frame

Units:

On the robot and the data sent from app:

- x: meters
- y: meters
- yaw: radians (data from app only send x and y, yaw must be infer from them)

- Vx: m/s
- Vy: m/s
- Wz: rad/s

On the app:

- x: percentage
- y: percentage

Touch input conversion

Absolute position (touch):

```
double absoluteX = rawX - boundaryLeft;
double absoluteY = rawY - boundaryTop;
```

- Converts raw screen coordinate to coordinate relative to *boundaryView*
- Clamped to boundary edges

Normalized position (0-1):

```
double normalizedX = absoluteX / screenWidth;
double normalizedY = absoluteY / screenHeight;
```

- Represents position as fraction of the screen

Percentage position (0-100%):

```
double percentageX = normalizedX * 100.0;
double percentageY = normalizedY * 100.0;
```

- Used as intermediate format for calculation into world meters
- Checked against 0-100% boundary

Global meters (for UDP):

```
double targetX = (percentageX / 100.0) * GAME_FIELD_X;
double targetY = (percentageY / 100.0) * GAME_FIELD_Y;
```

- Converts to meters based on field dimension
- Result is sent to robot

Robot odometry (UI display):

```
double inputX = odometryData[0] * (screenWidth / GAME_FIELD_X);
double inputY = odometryData[1] * (screenHeight / GAME_FIELD_Y);
```

- Converts meters to pixels for UI display
- Clamped boundary using *validateBoundary()*

2.4.6 Control logic

PD controller

- **Position errors:**

computed in global frame

```
current_x_error_g = self.desired_x - self.current_x
current_y_error_g = self.desired_y - self.current_y

current_yaw_p_error = self.normalize_angle(self.desired_yaw - self.current_yaw)
```

- **Derivative errors:**

```
current_yaw_d_error = (current_yaw_p_error - self.previous_yaw_p_error)/self.dt
current_x_d_error = (current_x_error_g - self.previous_x_p_error)/self.dt
current_y_d_error = (current_y_error_g - self.previous_y_p_error)/self.dt
```

- **Complementary filtering:**

apply complementary filter on D term to smooth the movement

```
current_x_d_error = ALPHA_D * current_x_d_error + (1 - ALPHA_D) * self.previous_x_d_
↪error
current_y_d_error = ALPHA_D * current_y_d_error + (1 - ALPHA_D) * self.previous_y_d_
↪error
current_yaw_d_error = ALPHA_D * current_yaw_d_error + (1 - ALPHA_D) * self.previous_
↪yaw_d_error
```

- **PD controller:**

```
linear_vel_x_g = self.k_p_linear * current_x_error_g + self.k_d * current_x_d_error
linear_vel_y_g = self.k_p_linear * current_y_error_g + self.k_d * current_y_d_error
angular_vel_z_g = self.k_p_yaw * current_yaw_p_error + self.k_d_yaw * current_yaw_d_
↪error
```

- **Frame conversion:**

```
linear_vel_x_l =  math.cos(self.current_yaw) * current_x_error_g + math.sin(self.
↪current_yaw) * current_y_error_g
linear_vel_y_l = -math.sin(self.current_yaw) * current_x_error_g + math.cos(self.
↪current_yaw) * current_y_error_g
```

- Units must match: meters (x, y), radians (yaw), m/s (velocities)
- Linear velocity in robot frame, angular velocity around z-axis
- Low-pass filtering applied on derivative term
- Return *linear_vel_x*, *linear_vel_y*, and *angular_vel_z*

Dead reckoning

If the velocity is less than error threshold, overwrite it with 0

Speed limit

If velocity exceeds maximum speed, overwrite it with maximum speed

2.4.7 Path planning

Arrivel detection

- Target reached if: - Distance < `goal_tolerance_pos = 0.05m` - Remains withitn tolerance for `0.5s`
- Arrival time accumulated using `dt` from timer callback

Waypoint callback

- Store each waypoint in a dictionary
- Append it into the queue
- Copy the waypoint in the queue to `active_target` to execute

Waypoints execution

- if `active_target` does not exist, and there's waypoint in the queue, pop the queue, then reset robot state to go to the waypoint that's being popped
- if return is triggered, and there's visited waypoint, command the robot to go the visited waypoint
- Otherwise, stop the robot

Waypoint update

- Store the edited waypoint in a dictionary
- If the edited waypoint is active, use complementary to smooth the robot movement when update to prevent jumping suddenly

```
self.active_target['x'] = (1 - ALPHA) * self.active_target['x'] + ALPHA * update_wp[
↪'updated_x']
self.active_target['y'] = (1 - ALPHA) * self.active_target['y'] + ALPHA * update_wp[
↪'updated_y']
```

- Otherwise, just update the waypoint as normal

Pause

- Robot pauses motion at each target
- If distance from the robot to the target is less than tolerance, stop the robot
- If the elapsed time during pause exceeds pause duration, command the robot to go to the next waypoint

Return

- Update the return flag

2.4.8 Odometry handling

Frame conversion

- Calculate velocity in local frame

```
linear_vel_x_local = (self.R/4) * (self.motor_vel[0] + self.motor_vel[1] + self.motor_
↪vel[2] + self.motor_vel[3])
linear_vel_y_local = (self.R/4) * (-self.motor_vel[0] + self.motor_vel[1] + self.
↪motor_vel[2] - self.motor_vel[3])
w_z = (self.R * (-self.motor_vel[0] + self.motor_vel[1] - self.motor_vel[2] + self.
↪motor_vel[3]))/(2 * (self.l_x + self.l_y))
```

- Convert them to global frame

```
v_x_global = math.cos(self.current_yaw) * linear_vel_x_local - math.sin(self.current_
↪yaw) * linear_vel_y_local
v_y_global = math.sin(self.current_yaw) * linear_vel_x_local + math.cos(self.current_
↪yaw) * linear_vel_y_local
```

- Integrate velocity to get position

```
self.x += v_x_global * self.dt
self.y += v_y_global * self.dt
```

Offset

```
self.x_odom = (self.x - self.x_start) - self.frame_offset_x
self.y_odom = (self.y - self.y_start) - self.frame_offset_y
```

Yaw

- Calculate yaw from quaternion

```
def calculate_yaw(self, q_x, q_y, q_z, q_w):
    siny_cosp = 2 * (q_w * q_z + q_x * q_y)
    cosy_sinp = 1 - 2 * (q_y**2 + q_z**2)
    return math.atan2(siny_cosp, cosy_sinp)
```

- Normalize yaw

```
def normalize_angle(self, a):
    return math.atan2(math.sin(a), math.cos(a))
```

- Offset yaw

```
self.current_yaw = (self.yaw - self.yaw_start) + 2 * math.pi * self.overflow_counter
```

- Convert to quaternion to publish

```
def publish_yaw(self, yaw):
    q = Quaternion()
    q.w = math.cos(yaw/2)
    q.x = 0.0
    q.y = 0.0
    q.z = math.sin(yaw/2)
    return q
```

2.4.9 Inverse Kinematic

```
V_wheel = [
Vx/self.R - Vy/self.R - omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R + Vy/self.R + omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R + Vy/self.R - omega_z*(self.lx + self.ly)/(2.0*self.R),
Vx/self.R - Vy/self.R + omega_z*(self.lx + self.ly)/(2.0*self.R)
]
```

2.5 Troubleshooting Guide

2.5.1 Communication issues

Symptom: robot doesn't repond to app command

Wi-Fi connectivity issues:

- Check that the robot is connected to the correct Wi-Fi network and has a valid IP address
- Verify IP address of the robot matches the one configured in the app
- Use `ifconfig` on the Jetson to check network interfaces and IP configuration

UDP packets not received:

- Confirm the correct port (default `5050`) is used
- Ensure packets are sent in **Little Endian** format

Firewall blocking UDP:

- Disable firewall or allow UDP traffic on port 5050

Symptom: waypoints not updating/edits ignored

Possible causes and solutions:

- Plan ID mismatch between app and robot
- Edited waypoint index out of range
- Packet corruption: check Little Endian encoding and packet size

2.5.2 Motor/driver issues

Symptom: robot does not move or moves incorrectly

Possible causes and solutions:

- Motor driver fuse blown: replace fuse and test
- Motors not powered: verify 24 V motor battery and DC-DC converter
- Encoder feedback lost: check encoder connections to STM32
- Wheels misaligned: check mecanum wheel orientation
- Incorrect velocities from controller: verify PD controller gains

Symptom: robot moves jerkily or overshoots targets

- Derivative term too noisy: verufy `ALPHA_D` filtering
- Low update frequency: check odometry reception at 10 Hz
- Active waypoint updates cause jumps: ensure complementary smoothing is applied

2.5.3 Sensor issues

Symptom: odometry is incorrect/robot drifts

- Checks IMU calibration and orientation
- Verify encoder counts correspond to expected wheel rotation
- Confirm yaw offset calculation (`yaw_start`, `overflow_counter`)
- Dead reckoning integration might accumulate error: check `dt` values

Symptom: robot reports zero movement despite motor command

- Motor velocities not reaching STM32: verify CAN bus communication
- Verify low-level motor control code maaps velocities correctly to wheels

2.5.4 App issues

Symptom: robot position not displayed correctly

- Verify coordinate conversions from meters to pixels (`metersToPxX/Y`) are correct
- Check `latestOdometry` is being updated from UDP socket
- Confirm screen boundary and field dimensions (`GAME_FIELD_X/Y`) match robot workspace

Symptom: touch input does not map to correct target

- Check touch, normalized, percentage, global meter conversion
- Ensure `boundaruView` dimensions match the UI

2.5.5 Safety/emergency stop

Symptom: robot does not stop when emergency button pressed

- Verify emergency stop wiring and fuses are intact
- Ensure software checks for emergency stop signal are active
- Test manually by cutting power to confirm hardware response

2.5.6 PD controller/motion control

Symptom: robot oscillates around waypoint

- Decreases `k_p_linear` or `k_p_yaw` gains
- Increase derivative damping (`ALPHA_D`)
- Verify derivative errors are computed correctly using `dt`

Symptom: robot fails to reach waypoint

- Check `goal_tolerance_pos` and pause duration
- Confirm `active_target` queue is populated and not empty
- Make sure velocity limits allow the robot to reach target within constraints

2.5.7 Low-level hardware checks

- **Jetson:** check power LED, CPU usage, and fan
- **STM32:** verify firmware is running; check CAN message counts
- **Batteries:** measure voltage under load; replace if below threshold
- **Motor drivers:** check for overcurrent events or thermal shutdowns

> **Attention**
>
> 1. Always start troubleshooting from connectivity, sensors, motion, control logic
> 2. Use logs from app (`sendWaypoints()`, `latestOdometry`) to verify packet flow
> 3. Test motor and wheel functionality before running autonomous commands
> 4. Keep fuses, emergency stop, and emergency protocols ready during testing

2.6 Update Log

2.6.1 V1.0

Features

Target_Setter app

- Basic waypoints coordinate
- Communicate with robot over UDP, JSON array packets

- Send waypoint one by one (when user adds one waypoint, send automatically)
- Implemented virtual joystick

> **Attention**
>
> - This version has no edit and return functionalities
> - No emergency stop is implemented on software

ROS2 packages

- PD controller with complementary filtering for translation
- P controller with velocity saturation for rotation
- Communicate with app at 10Hz over UDP
- Basic waypoint controller
- Feedback odometry to app
- Deadzone is implemented

> **Attention**
>
> No emergency stop is implementedon software layer

Performance

- Communication is sufficient
- PD controller is effective
- Yaw drifts due to mistakes in localization

2.6.2 V2.0

Updates

Target_Setter app

- Removed virtual joystick
- Communication is updated to binary packet with Little Endian byte order
- Definition of new packet structures
- Implemented edit and return functionalities
- Send waypoints only when user triggered

ROS2 packages

- Removed redundent nodes
- Added packet validation checked
- PD controller with complementary filtering
- Arrival detection added
- Added emergency stop

Performance

- Communication is efficient
- Yaw drift
- Controller is effective
- Undefined state behavior occurred

2.6.3 V3.0

Updates

ROS2 packages

- Corrects inverse kinematic equation
- Implements wheel odometry instead is using forward kinematic for odometry
- Simplifies controller to P controller with velocity saturation and rate limiting
- Improves arrival detection and planning logic
- Implements state machine

> **Attention**
>
> This version doesn't have any updates on Target_Setter app

CHAPTER

THREE

SYSTEM EVALUATION

3.1 test_session_1 documentation

3.1.1 Overview

Purpose

This documentation presents the stability analysis of the **Session 1** - the first system-level evaluation of the `target_setter` project. The goal is to characterize motor speed atbility and yaw behavior under two test conditions.

System under test

- **Platform:** 4-wheel omnidirectional robot
- **Controller:** `robot_control` (ROS 2 node) take waypoints from Android app `target_setter`
- **Sensors:** IMU (quaternion orientation), motor encoders (per-motor speed)
- **Communication:** CAN bus via `can_driver`, UDP via `udp_listener` and `udp_sender`
- **Odometry:** `current_odometry` node (fuses encoder + IMU)

Test conditions

Test	Description	Environment
TEST1	Curve-path at higher speed	Assume planar, indoor environment
TEST2	Lower speed with yaw commands	Assume planar, indoor environment

Data Collected

- ROS 2 log output containing: - Per-motor speed vectors (4 components each) - Odometry publishes: velocity (vx, vy) and orientation quaternion - `MotorCommand` publishes with `can_id` and `goal`
- Raw log file: `test_data/data_session_1.txt`

Signals Analyzed

1. **Overall speed:** mean of per-motor speed magnitudes at each timestamp
2. **Yaw rate (wz):** derivative of unwrapped yaw angle from quaternion
3. **Per-motor speed:** individual motor magnitude tracks for symmetry checks

Document Structure

Motor Speed Analysis	Per-motor speed traces, inter-motor symmetry
Overall Speed	Aggregated speed stability (rolling mean ± std)
Yaw Analysis	Yaw rate stability, command-response overlay
comparison	TEST1 vs TEST2 side-by-side
Conclusion	Summary of findings, issues observed, and next steps

3.1.2 Motor Speed Analysis

Objective

Examine individual motor speed traces to check for:

- **Data completeness** — are all 4 motors (CAN 102–105) reporting speed feedback?
- **Symmetry** — are all motors contributing equally?
- **Saturation** — does any motor hit its maximum and flatten out?
- **Oscillation** — is the speed steady or hunting?

Method

Each motor publishes a 4-component speed vector via CAN bus. The speed magnitude is computed as:

$$|\mathbf{v}_i| = \sqrt{s_1^2 + s_2^2 + s_3^2 + s_4^2}$$

The system has 4 motors on CAN IDs 102–105:

CAN ID	Motor	Speed data in log?	TEST1 samples	TEST2 samples
102	motor0	**No** — ACK only, no speed vector	0	0
103	motor1	Yes	53	77
104	motor2	Yes	8	20
105	motor3	Yes	43	2

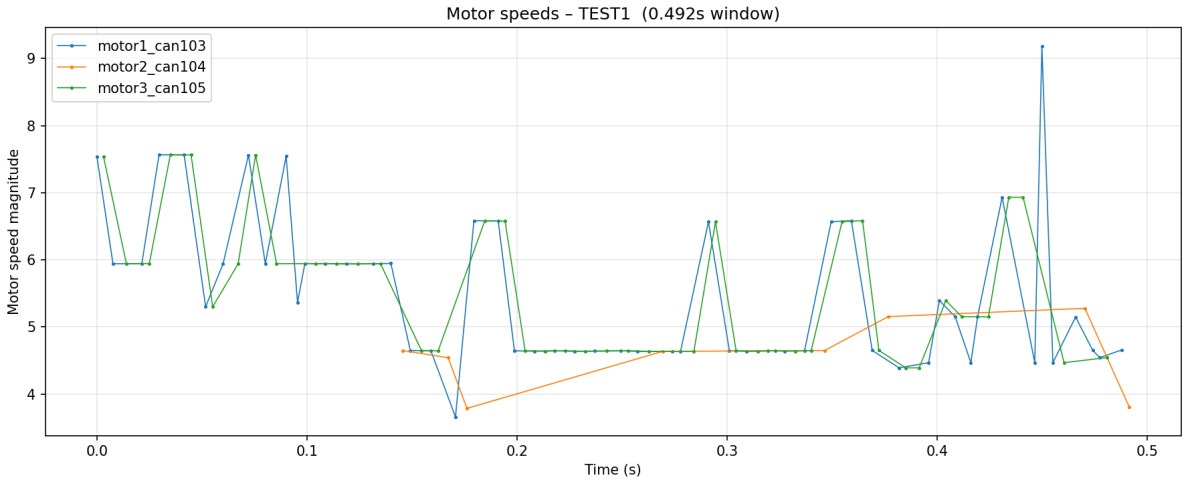
Warning

Motor 0 (CAN 102) is missing from all plots. The raw log contains `motor id: 0, can id = 102` acknowledgment lines, confirming the CAN message is received, but the `current_odometry` node never produces a `motor speed: [...]` line for it. The CAN frame arrives but is not correctly decoded — this is the `can_id` conflict.

Note

Motors do not publish at the exact same timestamps. Each motor reports independently, so traces appear interleaved and the per-motor sample counts are uneven.

TEST1



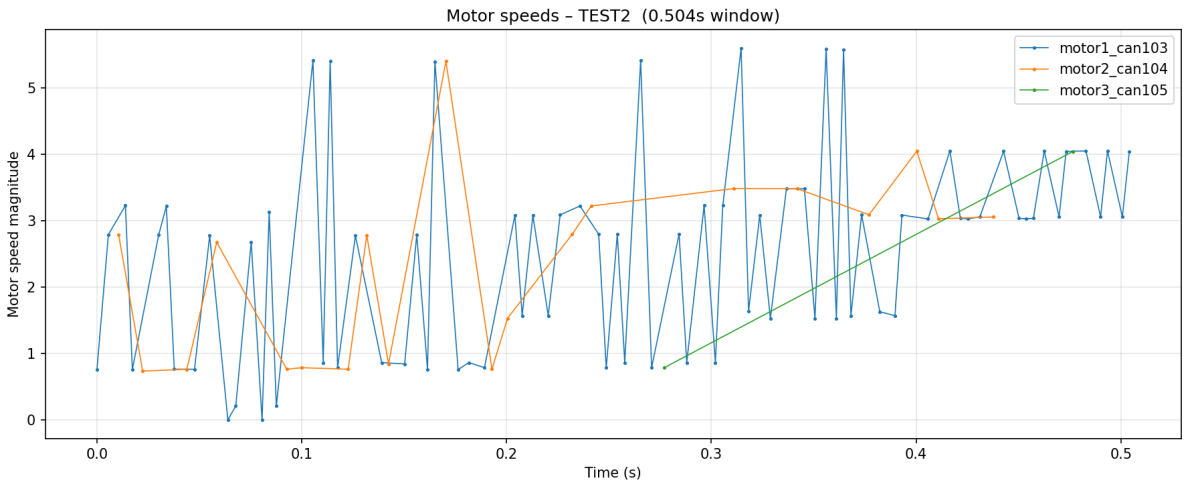
Observations:

- Data is **sparse** — only a handful of bursts over the 0.5 s window.
- **motor1_can103** (blue) dominates with 53 samples: starts at ~7.5, drops, then spikes to a **peak of ~9.2** near t = 0.45 s.
- **motor3_can105** (green) tracks motor1 closely when it appears (mean 5.53 vs 5.48), indicating these two motors are well-matched.
- **motor2_can104** (orange) has only 8 data points and a lower mean of **4.56** — approximately 17% slower than motors 1 and 3.

Per-motor mean speed:

Motor	Mean magnitude	Samples
motor1_can103	5.48	53
motor2_can104	4.56	8
motor3_can105	5.53	43

TEST2



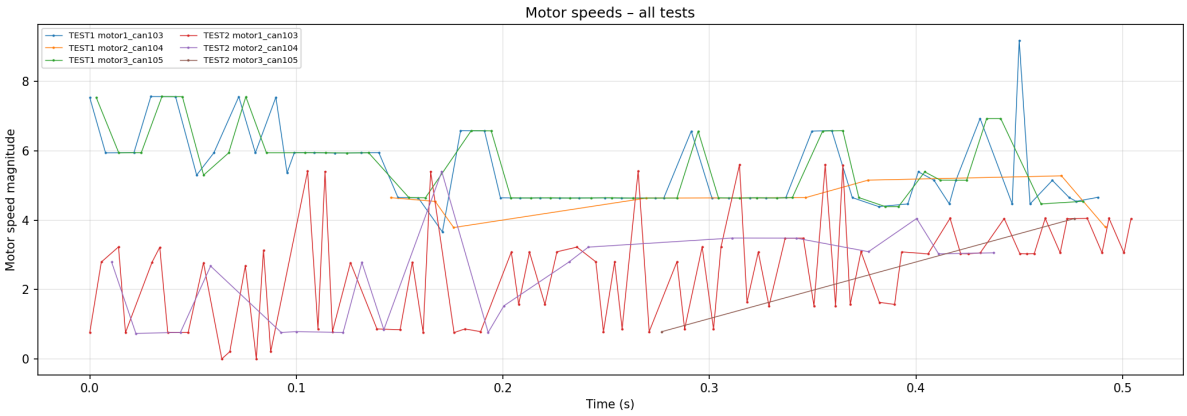
Observations:

- **motor1_can103** (blue) dominates with 77 samples and exhibits heavy **oscillation** — speed swings repeatedly between ~0 and ~5.6.
- **motor2_can104** (orange) appears briefly around t = 0.08–0.1 s and t = 0.19–0.22 s at low speeds (~0.8). Only 20 samples total.
- **motor3_can105** (green) is nearly absent — only **2 data points** in the entire test.
- The oscillatory pattern in motor1 (the primary reporting motor) suggests the controller is **hunting**: overshooting the target speed, correcting to near zero, then overshooting again.

Per-motor mean speed:

Motor	Mean magnitude	Samples
motor1_can103	2.53	77
motor2_can104	2.34	20
motor3_can105	2.42	2

Combined View



Both tests on a shared timeline showing the transition from higher-speed (TEST1) to lower-speed with turning (TEST2).

Key Findings

1. **Motor 0 (CAN 102) never reports speed data.** The CAN frame is received but `current_odometry` fails to decode it. This must be resolved before any stability conclusion can be considered complete — the controller is operating without feedback from one of four motors.
2. **Motor 2 (CAN 104) reports inconsistently** — 8 samples in TEST1, 20 in TEST2. Its mean speed is ~17% lower than motors 1 and 3 in TEST1 (4.56 vs ~5.5), which contributes to the curved path.
3. **Motor 3 (CAN 105) nearly vanishes in TEST2** — only 2 data points. CAN bus contention or message filtering may be dropping its feedback when motor commands are more active (turning).
4. **TEST2 shows clear speed oscillation** in motor1 — swinging 0 to 5.6 repeatedly. This is consistent with an under-damped PID speed loop or command latency causing the controller to hunt.
5. **The "overall speed" metric is unreliable** because it averages only the motors that reported at each timestamp. With motor0 missing entirely and motors 2/3 reporting sparsely, the overall speed is effectively just motor1's speed.

See also

- *Overall Speed* — aggregated speed stability (with the above caveat)
- *Yaw Analysis* — yaw rate behaviour resulting from motor asymmetry